10/19/2026

BFS,DFS,UCS

Artificial Intelligence Lab 07

Instructor: Sir Mustajab

Submitted by: Wasif Ali

Reg id: 245187

Class: BSCS F24 B



Lab No 07

Lab Task 1:

1. *(Easy)* **Implement BFS** in a Python script `friend_finder.py` for a small social network. Represent the network as an adjacency list (dictionary of name → list of friends) and write `bfs_shortest_path(graph, start, goal)` that returns the *shortest* chain of friends connecting two people (fewest introductions needed), by extending the BFS queue to store the path travelled so far instead of just the node. Test it by finding the shortest connection between two people who are not direct friends.

Python – Input:

```
network = {
    "Ali": ["Bilal", "Sara"],
    "Bilal": ["Ali", "Zara"],
    "Sara": ["Ali", "Omar"],
    "Zara": ["Bilal", "Hira"],
    "Omar": ["Sara"],
    "Hira": ["Zara"],
}
bfs_shortest_path(network, "Ali", "Hira")

Sample Output:
Shortest chain from Ali to Hira: ['Ali', 'Bilal', 'Zara', 'Hira']
Number of introductions needed: 3
```

Mandatory Git/GitHub steps:

(a) Inside the `Lab-7` folder, run `git init`; add `friend_finder.py` and commit the first working version with `git add` and `git commit`.

(b) Add one more person and connection to the `network` dictionary; run `git diff` to review the change before staging it, then commit it.

Code:

```python
def bfs_shortest_path(graph, start, goal):

    visited = [start]
    queue = [[start]]

    while queue:

        path = queue.pop(0)
        node = path[-1]

        if node == goal:
            return path, len(path) - 1
```

```
        for neighbour in graph.get(node, []):
            if neighbour not in visited:
                visited.append(neighbour)
                queue.append(path + [neighbour])

    return None, 0

network = {
    "Ali": ["Bilal", "Sara"],
    "Bilal": ["Ali", "Zara"],
    "Sara": ["Ali", "Omar"],
    "Zara": ["Bilal", "Hira"],
    "Omar": ["Sara"],
    "Hira": ["Zara"],
}

path, introductions = bfs_shortest_path(network, "Ali", "Hira")
print(f"Shortest chain from Ali to Hira: {path}")
print(f"Number of introductions needed: {introductions}")
```

Output:



Lab Task 2:

2. *(Easy)* **Implement DFS** in a Python script `maze_explorer.py` for a simple maze represented as an adjacency list of rooms. Write `dfs_can_reach(maze, start, exit_room)` that returns `True/False` for whether the exit is reachable from the start, and also prints the order in which rooms were visited while searching.

Python – Input:

```
maze = {
    "Entrance": ["Hall1", "Hall2"],
    "Hall1": ["Room1"],
    "Hall2": ["Room2", "Trap"],
    "Room1": [],
    "Room2": ["Exit"],
    "Trap": [],
    "Exit": [],
}
dfs_can_reach(maze, "Entrance", "Exit")

Sample Output:
Visit order: ['Entrance', 'Hall1', 'Room1', 'Hall2', 'Room2', 'Exit']
Can reach Exit from Entrance? True
```

Mandatory Git/GitHub steps:

(a) Add `maze_explorer.py` to the same `Lab-7` repository; run `git status` *before* staging to confirm it shows as untracked, then `git add` and `git commit`.

(b) Add one more room that is *not* reachable from the `Entrance`; run `git status` again after editing to see the modified file, then commit the change.

Code:

```python
def dfs_can_reach(maze, start, exit_room):
    visited = []

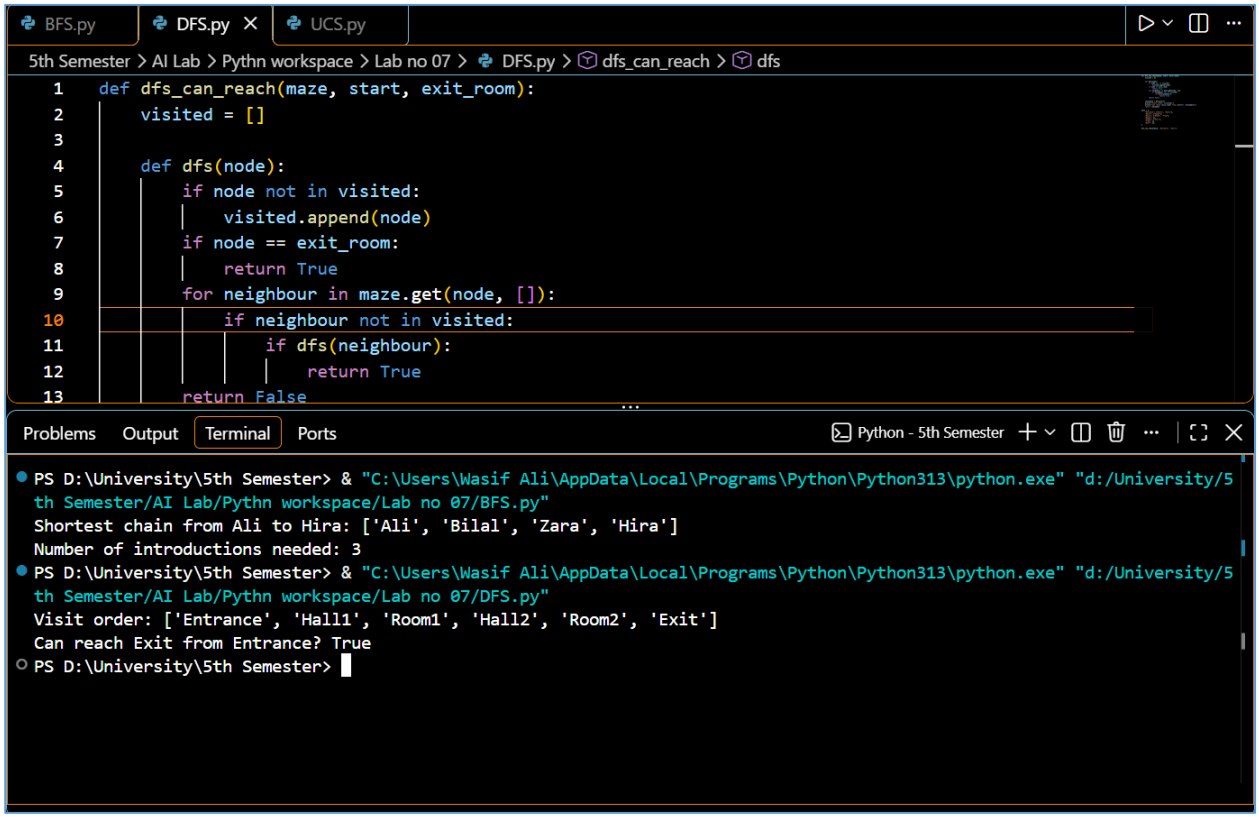
    def dfs(node):
        if node not in visited:
            visited.append(node)
        if node == exit_room:
            return True
        for neighbour in maze.get(node, []):
            if neighbour not in visited:
                if dfs(neighbour):
                    return True
        return False
```

```
    reachable = dfs(start)
    print(f"Visit order: {visited}")
    print(f"Can reach {exit_room} from {start}? {reachable}")
    return reachable

maze = {
    "Entrance": ["Hall1", "Hall2"],
    "Hall1": ["Room1"],
    "Hall2": ["Room2", "Trap"],
    "Room1": [],
    "Room2": ["Exit"],
    "Trap": [],
    "Exit": [],
}

dfs_can_reach(maze, "Entrance", "Exit")
```

Output:



Lab Task 3:

3. *(Medium)* **Implement UCS** in a Python script `flight_route_finder.py` that finds the cheapest flight route between two cities in a weighted graph of `(city, ticket_price)` connections. Write `ucs_cheapest_route(graph, start, goal)` returning both the route and its total cost, and also print *every* path explored (not just the final one) in the order UCS considers them, to show how the algorithm always expands the cheapest partial route first.

Python – Input:

```
flights = {
    "Islamabad": [("Lahore", 40), ("Karachi", 90)],
    "Lahore": [("Karachi", 50), ("Peshawar", 30)],
    "Karachi": [("Quetta", 60)],
    "Peshawar": [("Quetta", 70)],
    "Quetta": [],
}
ucs_cheapest_route(flights, "Islamabad", "Quetta")

Sample Output:
Cheapest route: ['Islamabad', 'Lahore', 'Peshawar', 'Quetta']
Total ticket cost: 140
```

Mandatory Git/GitHub steps:

(a) Add `flight_route_finder.py` to the `Lab-7` repository; commit the first working version with `git add` and `git commit`.

(b) Create and switch to a branch named `add-cost-report`; on that branch, add a short summary line printing how many routes were explored in total, then commit the change on that branch and switch back to `main` and `git merge` it in.

(c) Create a GitHub repository structured as `5th-Semester/AI-Lab/Lab-07`, add it with `git remote add origin`, and push your `Lab-7` folder – containing all three tasks – with `git push -u origin main`. Then `git clone` it into a second local folder to confirm everything downloaded correctly, make one more small change from that clone, and `git pull` it back into your original `Lab-7` folder.

Code:

```python
def path_cost(path):
    return sum(cost for _, cost in path)

def ucs_cheapest_route(graph, start, goal):
    visited = []
    queue = [[(start, 0)]]

    while queue:
        queue.sort(key=path_cost)
        path = queue.pop(0)

        route_nodes = [node for node, cost in path]
        current_cost = path_cost(path)
```

```
        print(f"Exploring route: {route_nodes} with cost: {current_cost}")

        node = path[-1][0]

        if node in visited:
            continue
        visited.append(node)

        if node == goal:
            return route_nodes, current_cost

        for neighbour, cost in graph.get(node, []):
            queue.append(path + [(neighbour, cost)])

    return None, None

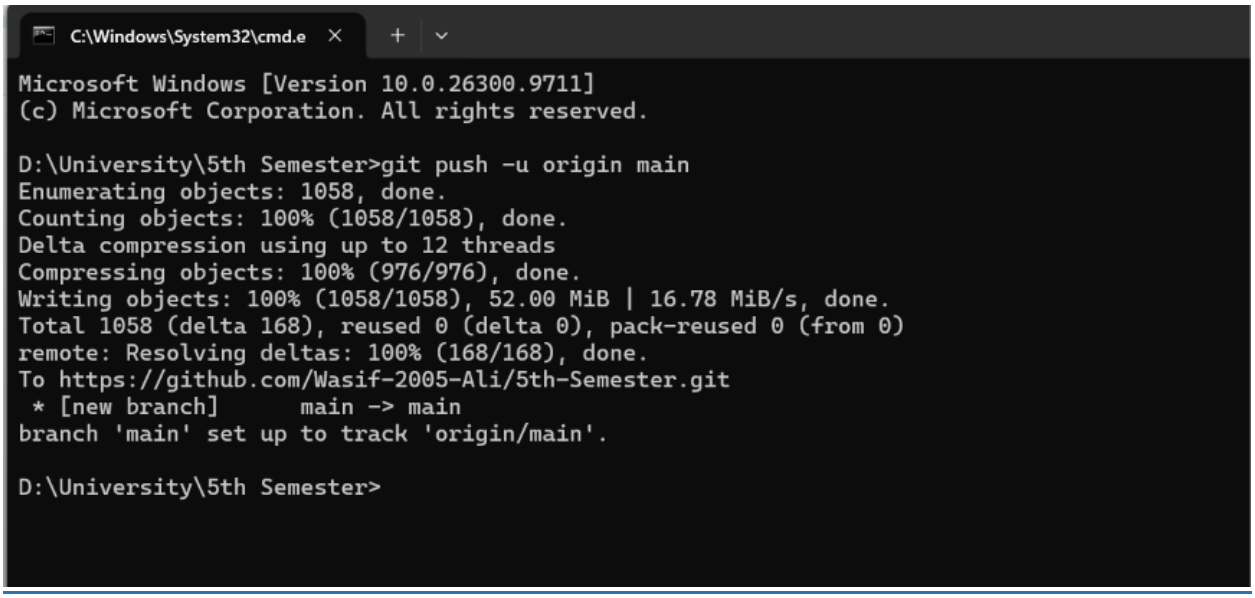
flights = {
    "Islamabad": [("Lahore", 40), ("Karachi", 90)],
    "Lahore": [("Karachi", 50), ("Peshawar", 30)],
    "Karachi": [("Quetta", 60)],
    "Peshawar": [("Quetta", 70)],
    "Quetta": [],
}

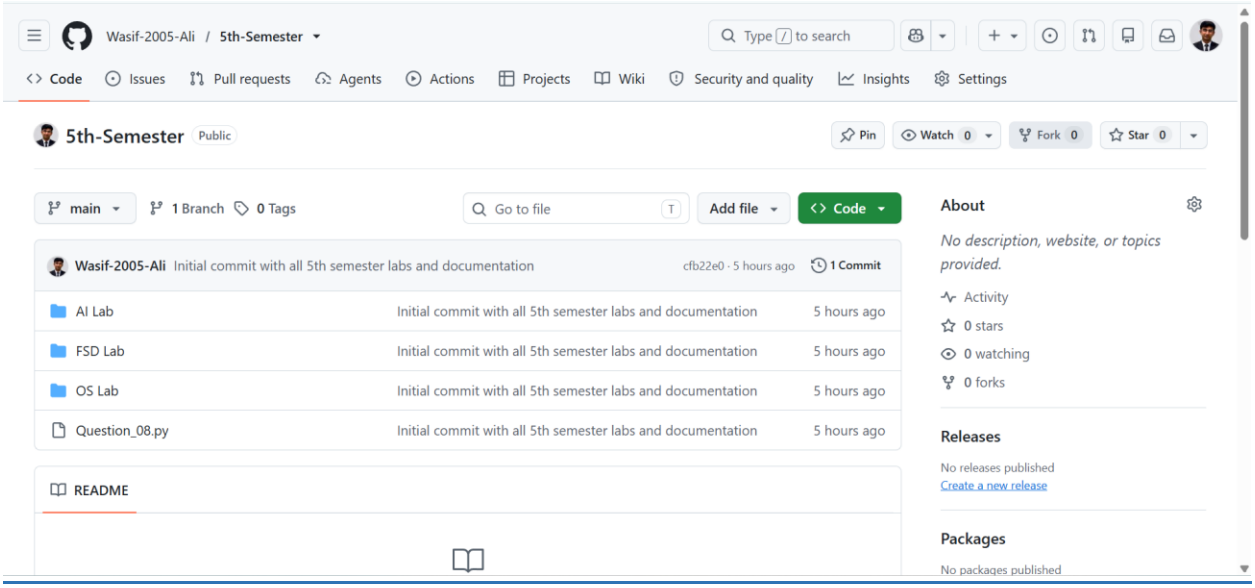
best_path, best_cost = ucs_cheapest_route(flights, "Islamabad", "Quetta")
print(f"Cheapest route: {best_path}")
print(f"Total ticket cost: {best_cost}")
```

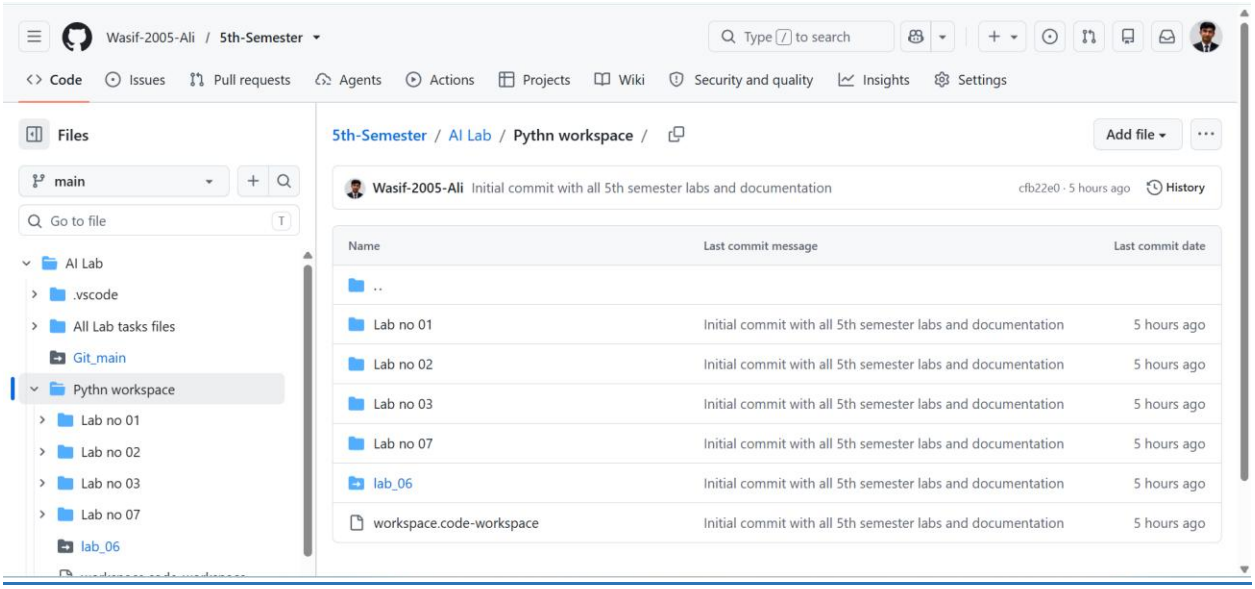
Output:

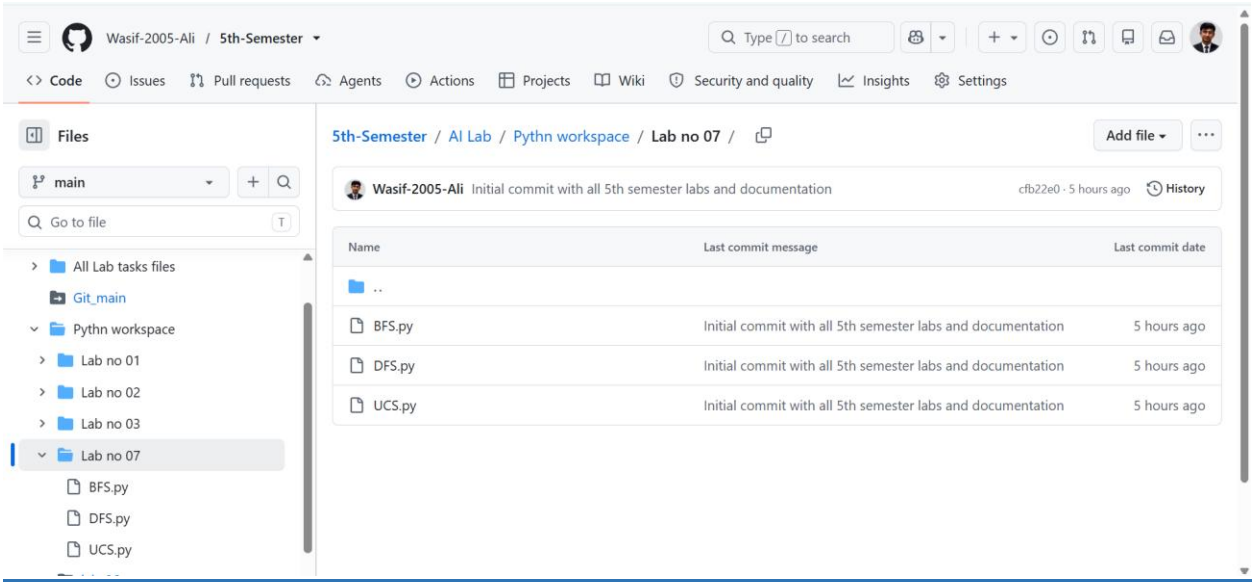
```
PS D:\University\5th Semester> & "C:\Users\Wasif Ali\AppData\Local\Programs\Python\Python313\python.exe" "d:/University/5
th Semester/AI Lab/Pythn workspace/Lab no 07/UCS.py"
Exploring route: ['Islamabad'] with cost: 0
Exploring route: ['Islamabad', 'Lahore'] with cost: 40
Exploring route: ['Islamabad', 'Lahore', 'Peshawar'] with cost: 70
Exploring route: ['Islamabad', 'Karachi'] with cost: 90
Exploring route: ['Islamabad', 'Lahore', 'Karachi'] with cost: 90
Exploring route: ['Islamabad', 'Lahore', 'Peshawar', 'Quetta'] with cost: 140
Cheapest route: ['Islamabad', 'Lahore', 'Peshawar', 'Quetta']
Total ticket cost: 140
PS D:\University\5th Semester>
```

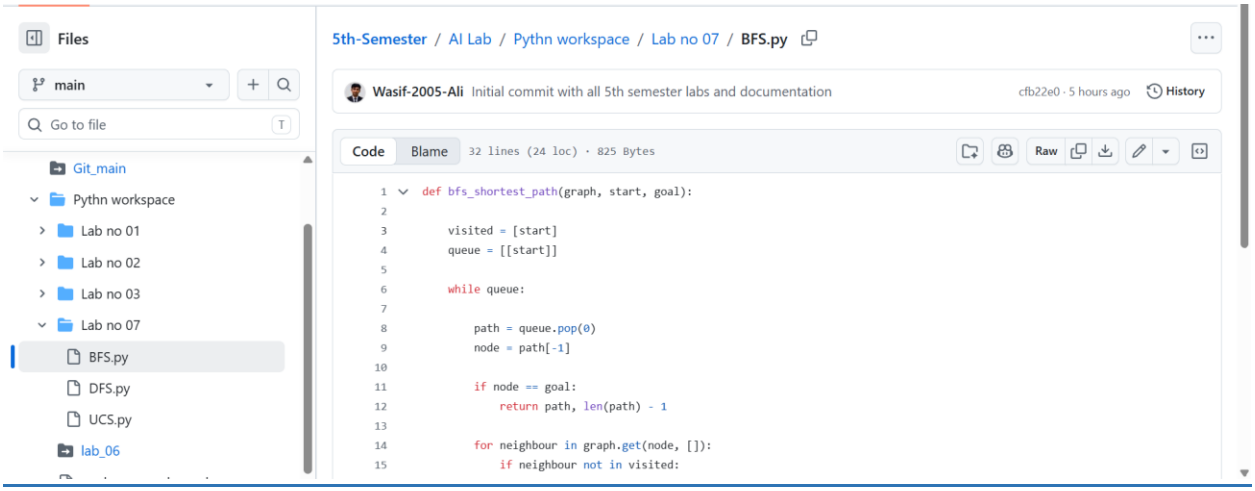
Git Hub Push:

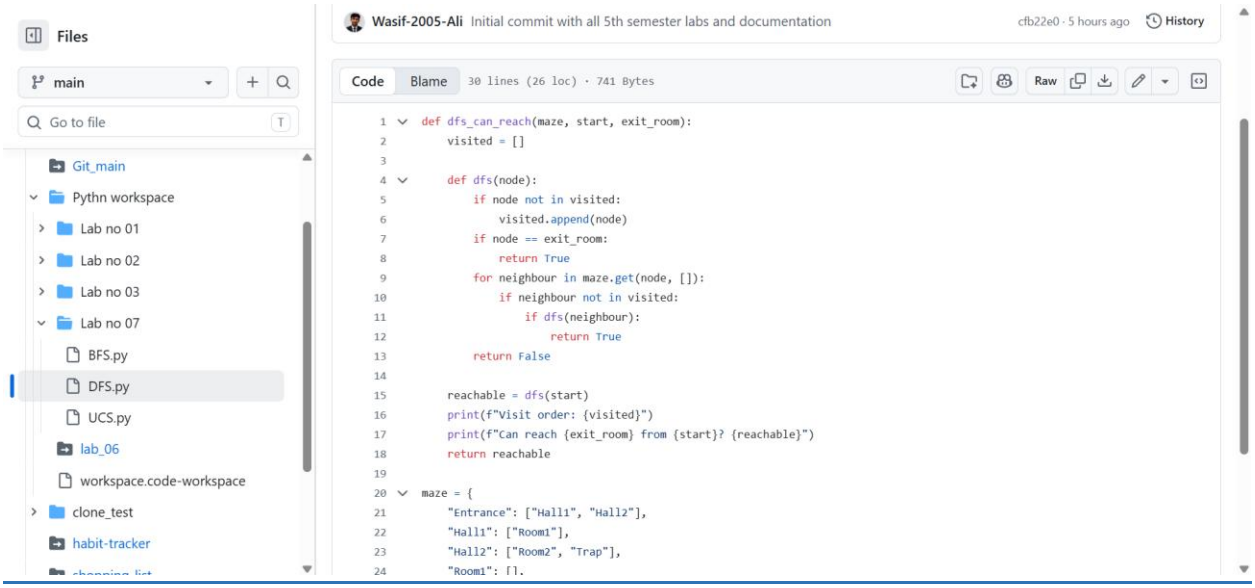

C:\Windows\System32\cmd.e
Microsoft Windows [Version 10.0.26300.9711]
(c) Microsoft Corporation. All rights reserved.
D:\University\5th Semester>git push -u origin main
Enumerating objects: 1058, done.
Counting objects: 100% (1058/1058), done.
Delta compression using up to 12 threads
Compressing objects: 100% (976/976), done.
Writing objects: 100% (1058/1058), 52.00 MiB | 16.78 MiB/s, done.
Total 1058 (delta 168), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (168/168), done.
To https://github.com/Wasif-2005-Ali/5th-Semester.git
* [new branch] main -> main
branch 'main' set up to track 'origin/main'.
D:\University\5th Semester>

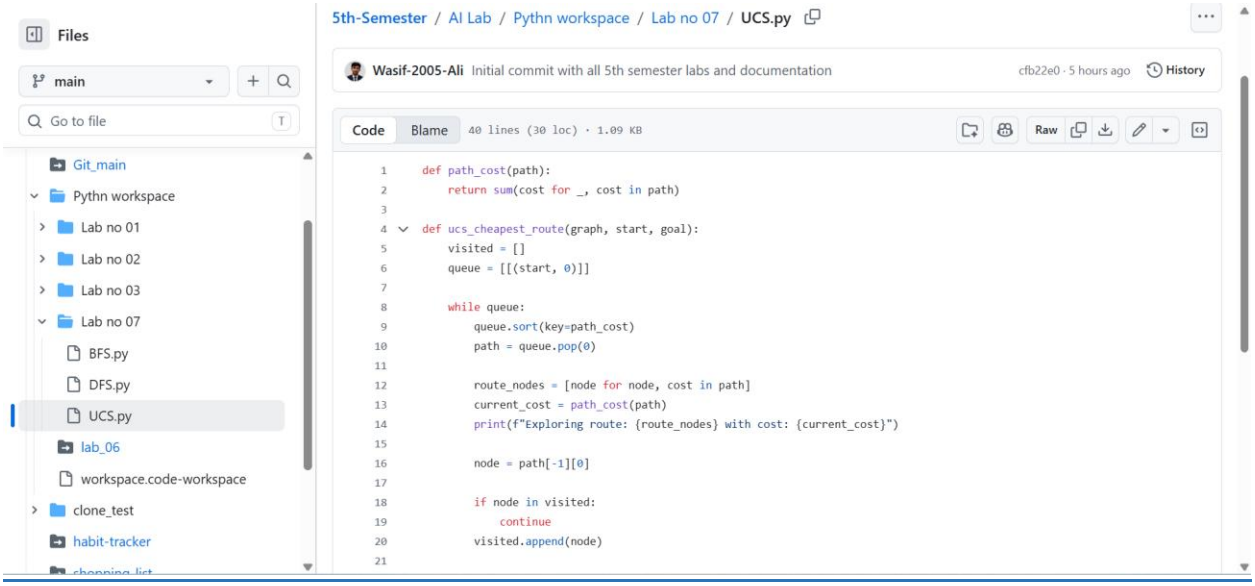

Wasif-2005-Ali / 5th-Semester
Type / to search
Code
Issues
Pull requests
Agents
Actions
Projects
Wiki
Security and quality
Insights
Settings
5th-Semester
Public
Pin
Watch 0
Fork 0
Star 0
main
1 Branch
0 Tags
Go to file
Add file
Code
Wasif-2005-Ali Initial commit with all 5th semester labs and documentation
cfb22e0 · 5 hours ago
1 Commit
AI Lab
Initial commit with all 5th semester labs and documentation
5 hours ago
FSD Lab
Initial commit with all 5th semester labs and documentation
5 hours ago
OS Lab
Initial commit with all 5th semester labs and documentation
5 hours ago
Question_08.py
Initial commit with all 5th semester labs and documentation
5 hours ago
README
About
No description, website, or topics provided.
Activity
0 stars
0 watching
0 forks
Releases
No releases published
Create a new release
Packages
No packages published


Wasif-2005-Ali / 5th-Semester
Type / to search
Code
Issues
Pull requests
Agents
Actions
Projects
Wiki
Security and quality
Insights
Settings
Files
main
Go to file
AI Lab
.vscode
All Lab tasks files
Git_main
Pythn workspace
Lab no 01
Lab no 02
Lab no 03
Lab no 07
lab_06
5th-Semester / AI Lab / Pythn workspace /
Add file
Wasif-2005-Ali Initial commit with all 5th semester labs and documentation
cfb22e0 · 5 hours ago
History
Name
Last commit message
Last commit date
..
Lab no 01 Initial commit with all 5th semester labs and documentation 5 hours ago
Lab no 02 Initial commit with all 5th semester labs and documentation 5 hours ago
Lab no 03 Initial commit with all 5th semester labs and documentation 5 hours ago
Lab no 07 Initial commit with all 5th semester labs and documentation 5 hours ago
lab_06 Initial commit with all 5th semester labs and documentation 5 hours ago
workspace.code-workspace Initial commit with all 5th semester labs and documentation 5 hours ago


Wasif-2005-Ali / 5th-Semester
Type / to search
Code
Issues
Pull requests
Agents
Actions
Projects
Wiki
Security and quality
Insights
Settings
Files
main
Go to file
All Lab tasks files
Git_main
Pythn workspace
Lab no 01
Lab no 02
Lab no 03
Lab no 07
BFS.py
DFS.py
UCS.py
5th-Semester / AI Lab / Pythn workspace / Lab no 07 /
Add file
Wasif-2005-Ali Initial commit with all 5th semester labs and documentation
cfb22e0 · 5 hours ago
History
Name
Last commit message
Last commit date
..
BFS.py Initial commit with all 5th semester labs and documentation 5 hours ago
DFS.py Initial commit with all 5th semester labs and documentation 5 hours ago
UCS.py Initial commit with all 5th semester labs and documentation 5 hours ago


Files
main
Go to file
Git_main
Pythn workspace
Lab no 01
Lab no 02
Lab no 03
Lab no 07
BFS.py
DFS.py
UCS.py
lab_06
5th-Semester / AI Lab / Pythn workspace / Lab no 07 / BFS.py
Wasif-2005-Ali Initial commit with all 5th semester labs and documentation
cfb22e0 · 5 hours ago
History
Code
Blame
32 lines (24 loc) · 825 Bytes
Raw
def bfs_shortest_path(graph, start, goal):
visited = [start]
queue = [[start]]
while queue:
path = queue.pop(0)
node = path[-1]
if node == goal:
return path, len(path) - 1
for neighbour in graph.get(node, []):
if neighbour not in visited:





ALL TASK COMPLETED SUCCESFULLY